Minimizzazione del makespan su macchine parallele $P_m||C_{\max}$

Confronto prestazionale tra Programmazione Dinamica, Modello PLI e Euristica LPT

Greta Luna Ancora (Matr. 0369455)

Corso di Algoritmi e Modelli per l'Ottimizzazione Discreta
Università di Roma Tor Vergata

Definizione del Problema

Classificazione: Minimizzazione del makespan su macchine parallele $P_m||C_{\max}$.

Sono dati n job indipendenti, ciascuno con tempo di processamento intero positivo $p_j \in \mathbb{Z}^+$. Non vi sono release date, due date o altri vincoli temporali.
Sono date m macchine parallele ed identiche.
Ogni job deve essere assegnato a 1! macchina, senza preemption.

Obiettivo: Minimizzare il makespan, cioè il massimo carico tra le macchine

$$C_{max} = \max_{h=1,\ldots,m} L_h$$

dove L_h indica il carico totale dei job assegnati alla macchina h.

Nota Teorica

Il problema si riduce a partizionare gli n job in m sottoinsiemi, minimizzando il massimo carico dei sottoinsiemi.
Il problema del partizionamento è un problema NP-completo.

Il caso di due macchine: $P_2||C_{\max}$

Con due macchine è sufficiente conoscere quali carichi possono essere ottenuti sulla prima macchina, poiché il carico della seconda è determinato implicitamente dal carico totale dei job già assegnati.

Definizione dello Stato:

- $R_i \subseteq \{0, \ldots, P\}$ è l'insieme dei carichi ottenibili sulla prima macchina utilizzando solo i primi i job.
- $P = \sum_{j=1}^{n} p_j$ è il tempo di processamento totale.

Condizione Iniziale e Transizione:

- $R_0 = \{0\}$.
- Il job i può essere assegnato:
  - alla prima macchina aumentando il suo carico di p_i;
  - alla seconda macchina lasciando invariato il carico della prima.
- L'equazione di ricorrenza è $R_i = R_{i-1} \cup \{s + p_i : s \in R_{i-1}\}$.

Soluzione Ottima

Alla fine, per ogni carico $s \in R_n$, il carico della seconda macchina è $P - s$.
Il valore ottimo è $C^*_{max} = \min_{s \in R_n} \max\{s, P - s\}$.

Generalizzazione a m macchine

Per $m > 2$, l'idea è mantenere esplicitamente i carichi delle prime $m-1$ macchine. Il carico della macchina m viene determinato implicitamente.

Definizione dello Stato:

- $R_i \subseteq \{0, \ldots, P\}^{m-1}$ è l'insieme dei vettori di carico raggiungibili sulle prime $m-1$ macchine con i primi i job.
- Uno stato ha la forma $(x_1, \ldots, x_{m-1})$, dove x_h è il carico assegnato alla macchina h, per $h = 1, ..., m-1$.

Condizione Iniziale e Transizione:

- $R_0 = \{(0, \ldots, 0)\}$.
- Il job i può essere assegnato:
  - a una delle prime $m-1$ macchine ottenendo, ad esempio, il nuovo stato $(x_1, ..., x_h + p_i, ..., x_{m-1})$, se decidiamo di assegnare il job i alla macchina h;
  - alla macchina m lasciando invariato il vettore.

L'equazione di ricorrenza è:

$$R_i = R_{i-1} \cup \bigcup_{h=1}^{m-1} \{(x_1, \ldots, x_h + p_i, \ldots, x_{m-1}) : (x_1, \ldots, x_{m-1}) \in R_{i-1}\}$$

Soluzione Ottima

Valutazione dell'Ottimo:
Alla fine, per ogni vettore raggiungibile $(x_1, \ldots, x_{m-1}) \in R_n$, il carico della macchina m si ottiene per sottrazione:

$$x_m = P - \sum_{h=1}^{m-1} x_h$$

Il valore ottimo è:

$$C^*_{max} = \min_{(x_1,\ldots,x_{m-1}) \in R_n} \max \left(x_1, \ldots, x_{m-1}, P - \sum_{h=1}^{m-1} x_h \right)$$

Analisi della Complessità

Complessità Temporale:
Il numero di vettori di carico possibili per le prime $m-1$ macchine è al più $(P+1)^{m-1}$. Per ciascun job e per ciascuno stato, si generano al più m transizioni. Pertanto, una stima diretta della complessità è:

$$\mathcal{O}\left(nmP^{m-1}\right)$$

Algoritmo Pseudo-polinomiale

Se il numero di macchine m è fissato, questa complessità è pseudo-polinomiale. Per esempio:

- $m = 2 \Rightarrow \mathcal{O}(nP)$
- $m = 3 \Rightarrow \mathcal{O}(nP^2)$
- $m = 4 \Rightarrow \mathcal{O}(nP^3)$

Complessità Spaziale:
La memoria richiesta può essere ridotta a $\mathcal{O}(P^{m-1})$ limitandosi a mantenere soltanto due livelli della programmazione dinamica, cioè R_{i-1} e R_i.

Il Modello Esatto PLI per $P_m||C_{\max}$

Variabili di Decisione e Funzione Obiettivo:

- $x_{ih} \in \{0, 1\}$: vale 1 se il job i è assegnato alla macchina h, 0 altrimenti.
- $C_{\max} \geq 0$: variabile continua che rappresenta il makespan globale.

Obiettivo: $\min C_{\max}$

Vincoli Strutturali:

- **Assegnamento unico:** Ogni job i è assegnato a una sola macchina.

$$\sum_{h=1}^{m} x_{ih} = 1 \quad \forall i \in \{1, \ldots, n\}$$

- **Definizione del makespan:** Il carico L_h di ciascuna macchina h non supera $C_{\max}$.

$$L_h = \sum_{i=1}^{n} p_i x_{ih} \leq C_{\max} \quad \forall h \in \{1, \ldots, m\}$$

Miglioramenti per le Prestazioni del Solver

Essendo le macchine identiche, il modello base presenta forti simmetrie matematiche: ogni assegnazione valida possiede $m!$ permutazioni equivalenti che ingigantiscono l'albero di ricerca del Branch-and-Bound.

Symmetry Breaking:

- **Vincolo:** Si impone un ordinamento decrescente sui carichi delle macchine: $L_1 \geq L_2 \geq \cdots \geq L_m$.
- **Vantaggio:** Forza il modello ad accettare una sola rappresentazione per ogni famiglia di soluzioni equivalenti. Questo taglia drasticamente i rami ridondanti e accelera la dimostrazione di ottimalità.

Lower Bound:

- **Vincoli:** $C_{\max} \geq \max_i p_i$ e $C_{\max} \geq \lceil \frac{1}{m} \sum_{i=1}^{n} p_i \rceil$.
- **Vantaggio:** Fornisce al solutore un "pavimento" matematico noto sotto il quale è impossibile scendere.

L'Euristica Longest Processing Time (LPT)

Principio e Logica:

- **Ordinamento preliminare:** I job vengono ordinati in maniera non crescente rispetto al tempo di elaborazione:

$$p_1 \geq p_2 \geq \cdots \geq p_n$$

- **Assegnamento Greedy:** Ciascun job viene assegnato iterativamente alla macchina che al momento ha il carico minore.

Procedura Algoritmica:

1. Ordina i n job in modo che $p_1 \geq p_2 \geq \cdots \geq p_n$.
2. Inizializza i carichi delle m macchine a zero: $L_h = 0 \ \forall h \in \{1, \ldots, m\}$.
3. **For** $i = 1$ to n:
   - Trova la macchina h^* con carico minimo: $h^* = \arg\min_h L_h$.
   - Assegna il job i alla macchina h^* e aggiorna il carico: $L_{h^*} \leftarrow L_{h^*} + p_i$.

Rapporto di Approssimazione:

$$\frac{C_{\max}^{\text{LPT}}}{C_{\max}^{*}} \leq \frac{4}{3} - \frac{1}{3m}$$

Design Sperimentale: Generazione e Parametri

Tutti i test base sono stati eseguiti impostando un time limit di 120 secondi e un budget di memoria di 16 GB.
Le classi sperimentali sono state generate combinando i seguenti parametri:

- **Numero di macchine** m**:** 2,3,5.
- **Numero di job** n**:** $n_{piccolo}, n_{medio}, n_{grande}$.
- **Varianza dei tempi di processamento** p_j**:** Bassa $\in [1, 100]$ vs Alta $\in [1, 1000]$.

Nei test di comparazione dei tre metodi sono state generate 10 istanze per ogni classe, per un totale di 180 istanze esaminate.

Comportamento e Scalabilità del Modello PLI

Analisi Preliminare del Solutore Esatto:

- È stata condotta una prima fase di esplorazione per valutare i limiti computazionali del modello PLI al crescere del numero di job n.
- **Risultato:** Il solutore MIP (Gurobi) ha mostrato un'eccellente capacità di scalare rispetto a n.
- I tempi di calcolo iniziano a saturare il time limit solamente per istanze enormi, con un numero di job nell'ordine di 10^6.

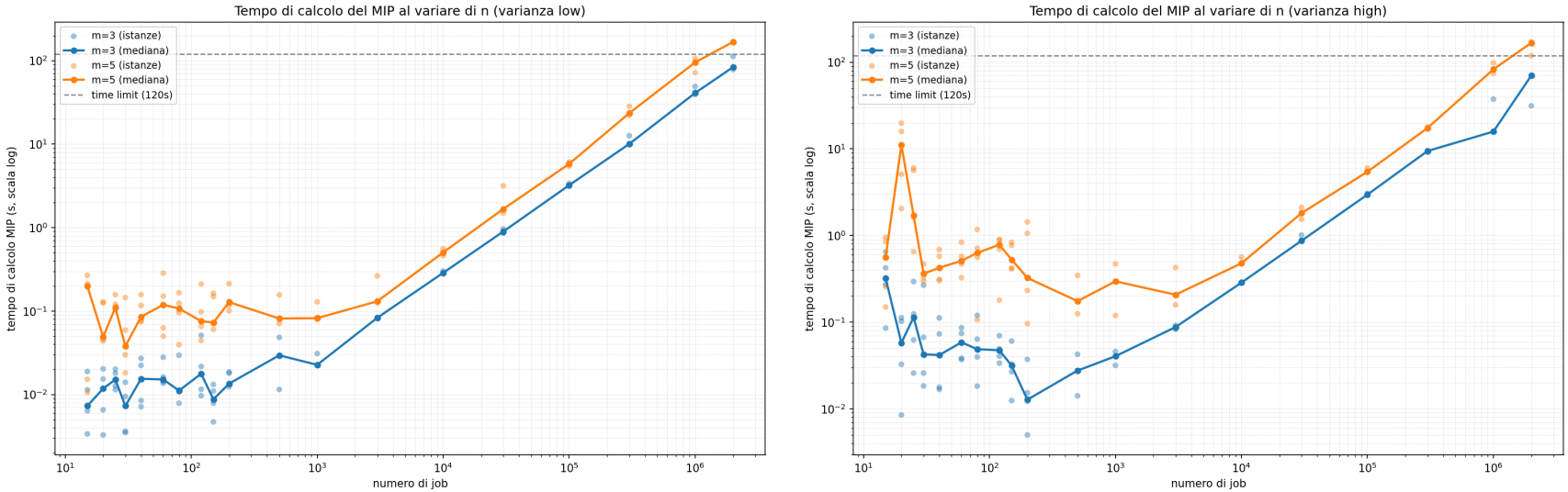


Figura: Andamento dei tempi del MIP.

Limiti della Programmazione Dinamica Base

Analisi Preliminare della DP: Parallelamente, è stato studiato l'andamento dei tempi di computazione e dell'occupazione di memoria per l'algoritmo di Programmazione Dinamica.

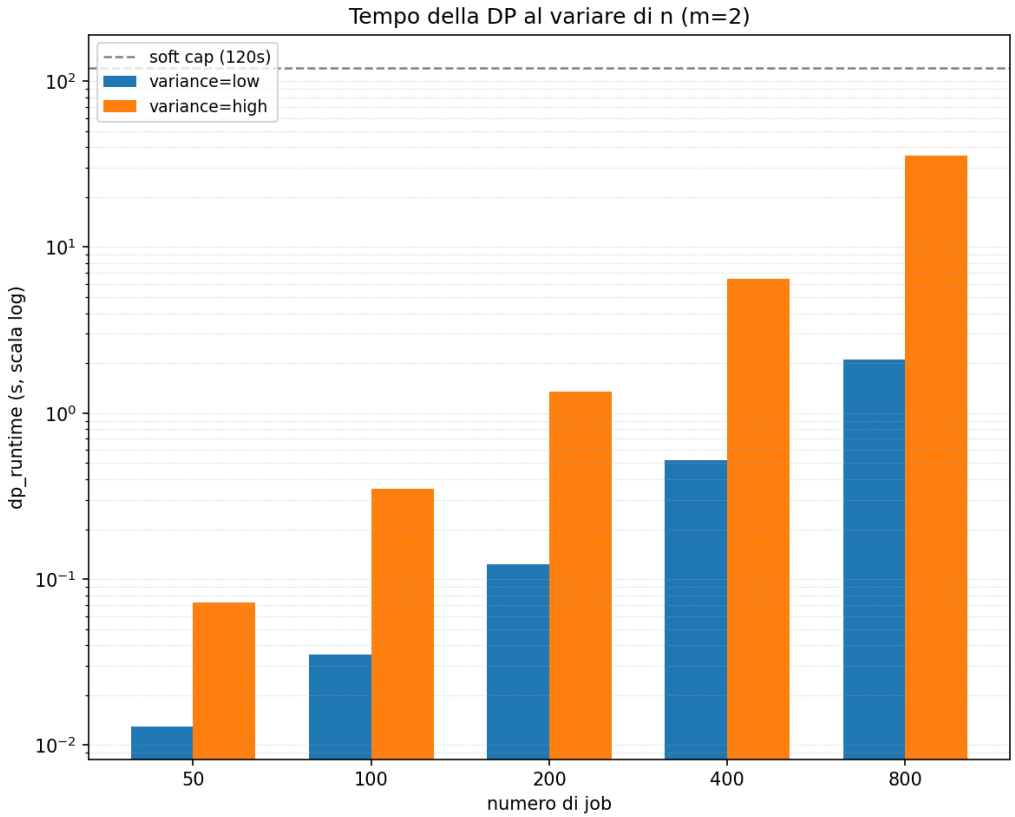


Figura: Andamento dei tempi della DP.

Limiti della Programmazione Dinamica Base

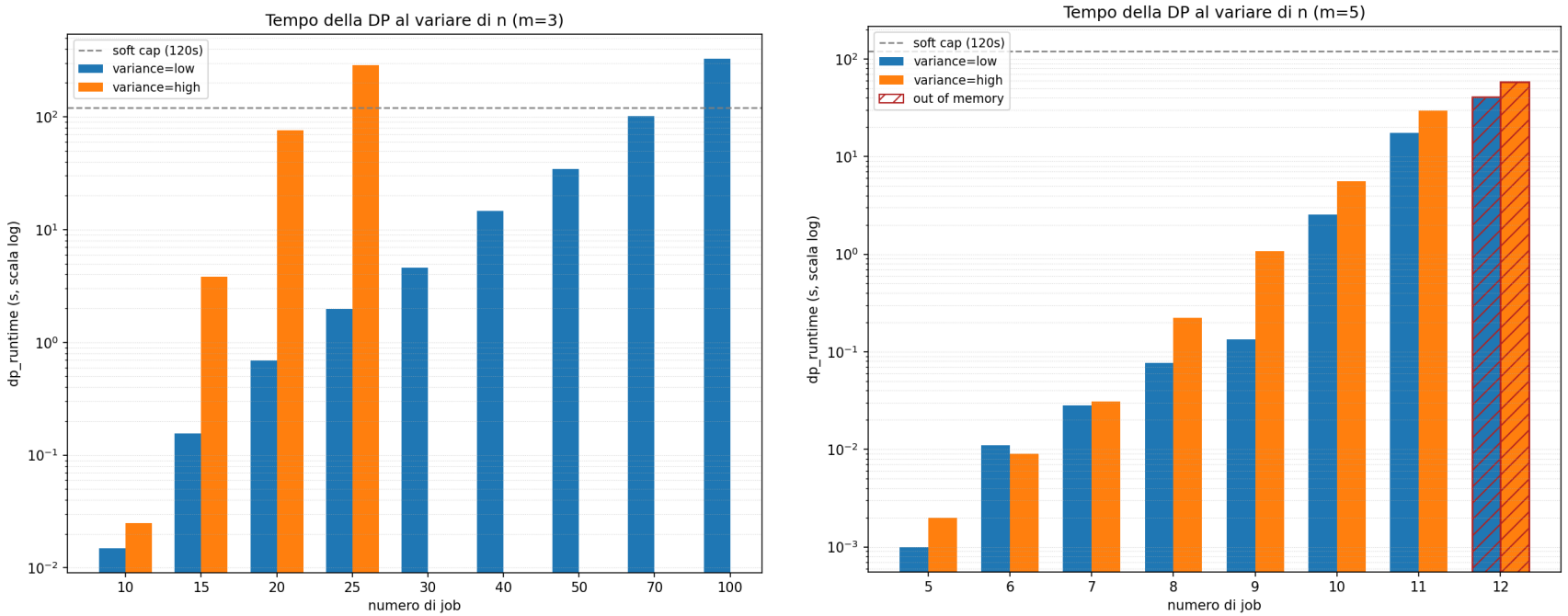


Figura: Andamento dei tempi della DP.

Limiti della Programmazione Dinamica

- **Risultato critico:** La DP va in *Out of Memory* (OOM) e fallisce molto prima del MIP: la tolleranza massima di n è inferiore di ben 4 ordini di grandezza.

Scelta delle Soglie Iniziali

A causa dei severi limiti spaziali e temporali della DP, per il primo esperimento comparativo sono state fissate delle classi di istanze molto ridotte:

$$n \in \{10, 25, 50\}$$

Primo Esperimento di Comparazione

Confronto MIP vs DP vs LPT:

- **Ottimalità:** Sia MIP che DP garantiscono la soluzione ottima (entro i limiti di memoria), mentre LPT restituisce soluzioni approssimate ma in tempi irrisori.
- **Collo di bottiglia della DP:** Si è osservato che, all'aumentare di n e della varianza dei tempi di processamento, i tempi della DP diventano onerosi.

Primo Esperimento di Comparazione

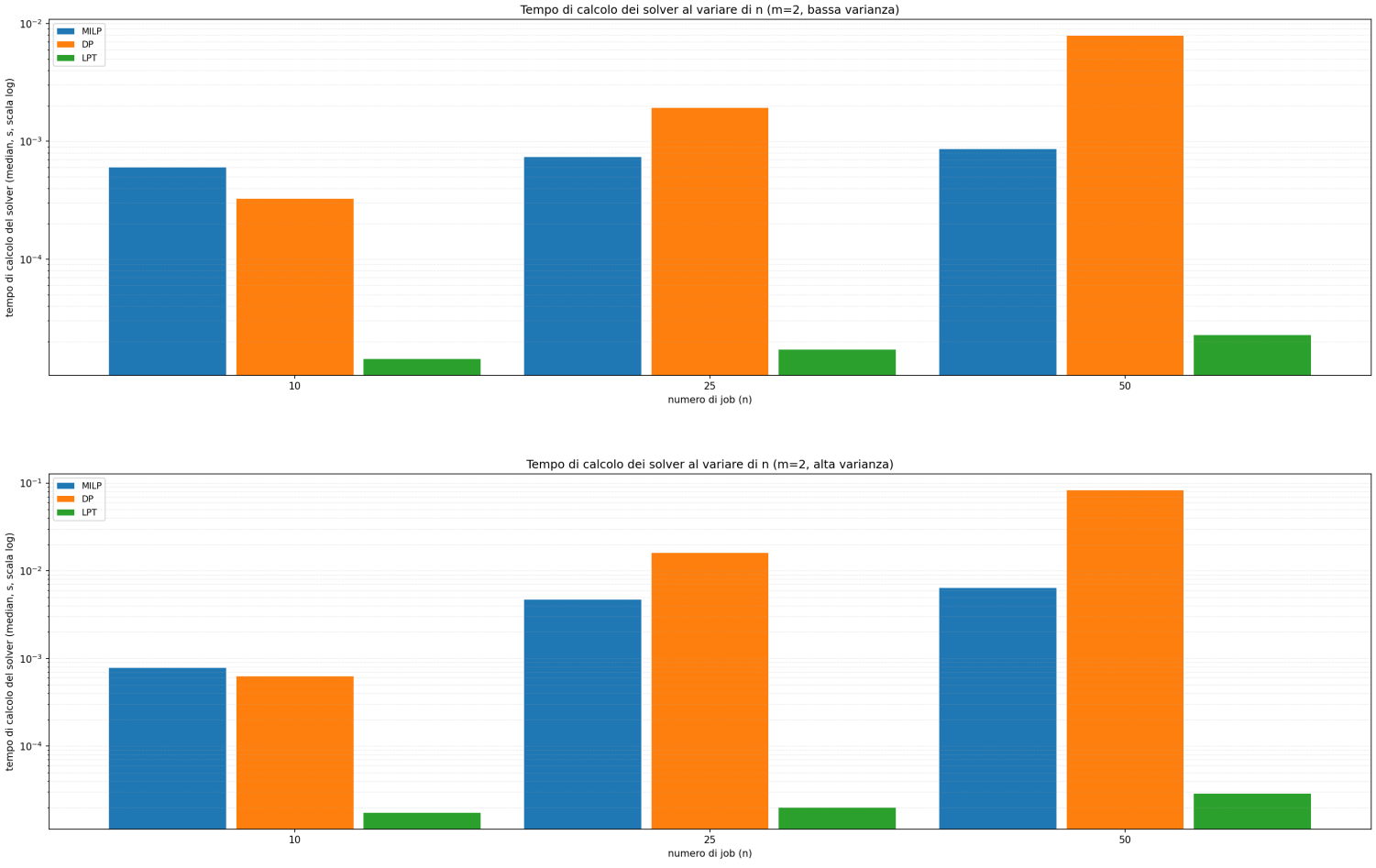


Figura: Tempi di esecuzione nel primo esperimento, m=2.

Primo Esperimento di Comparazione

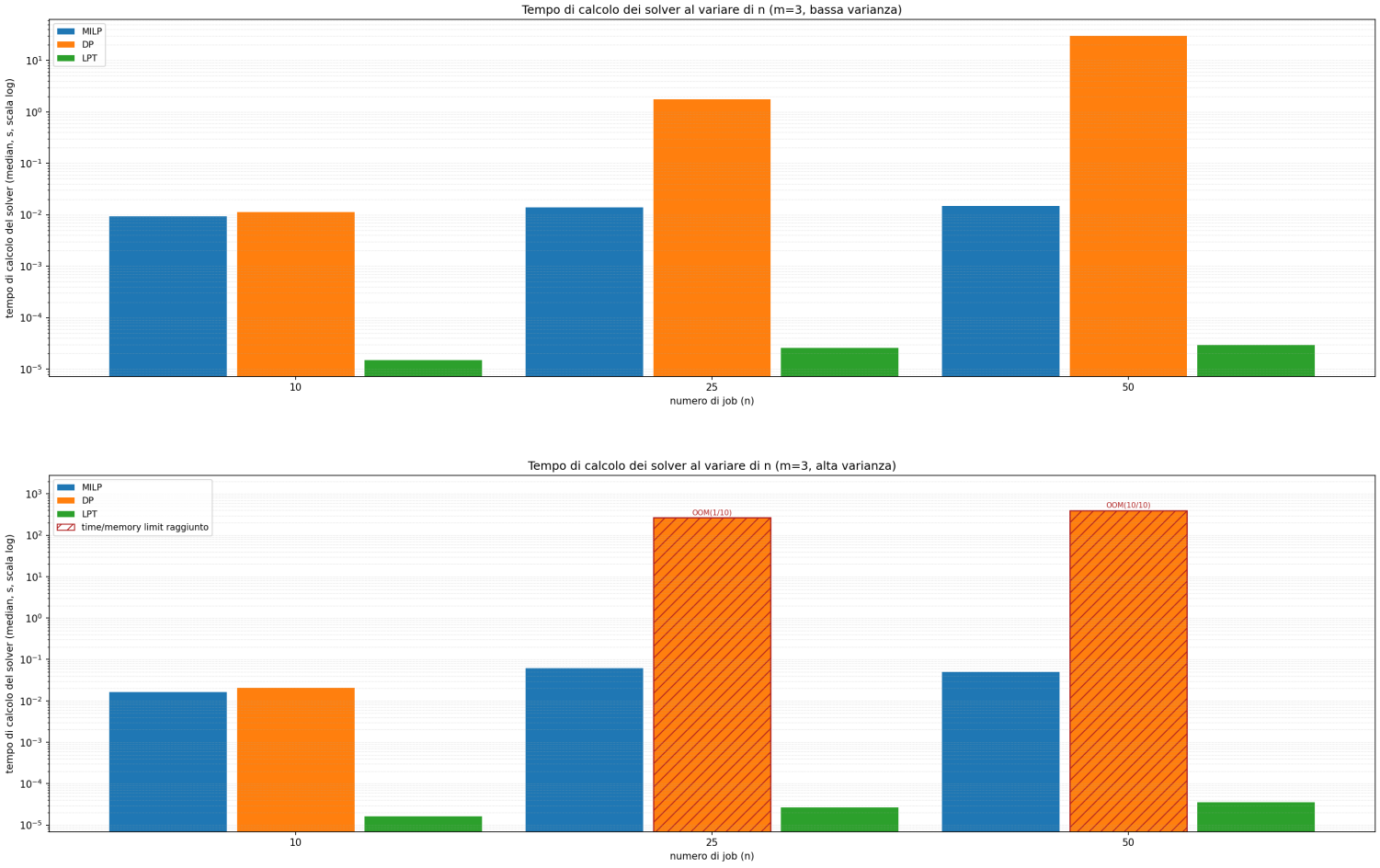


Figura: Tempi di esecuzione nel primo esperimento, m=3.

Primo Esperimento di Comparazione

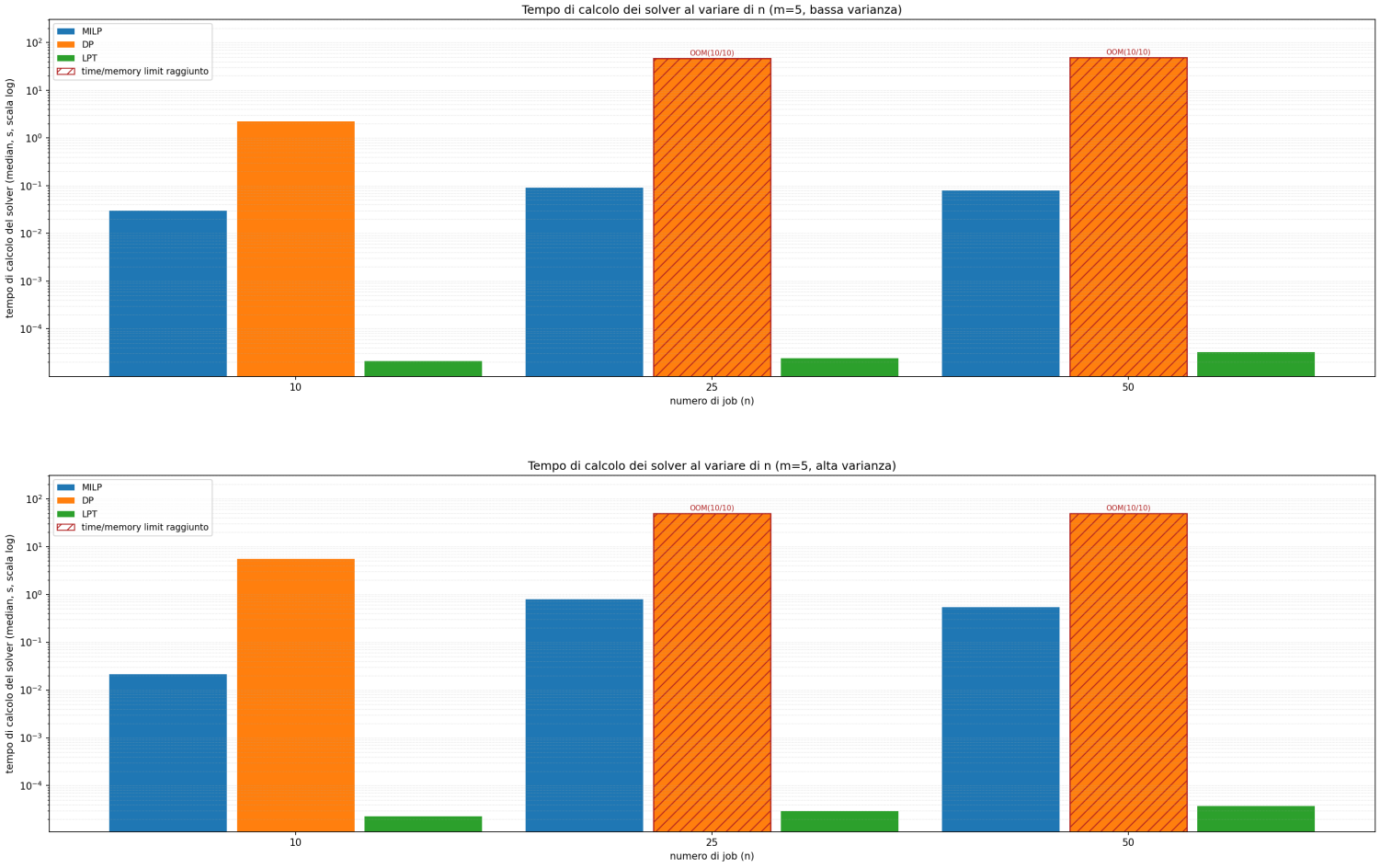


Figura: Tempi di esecuzione nel primo esperimento, m=5.

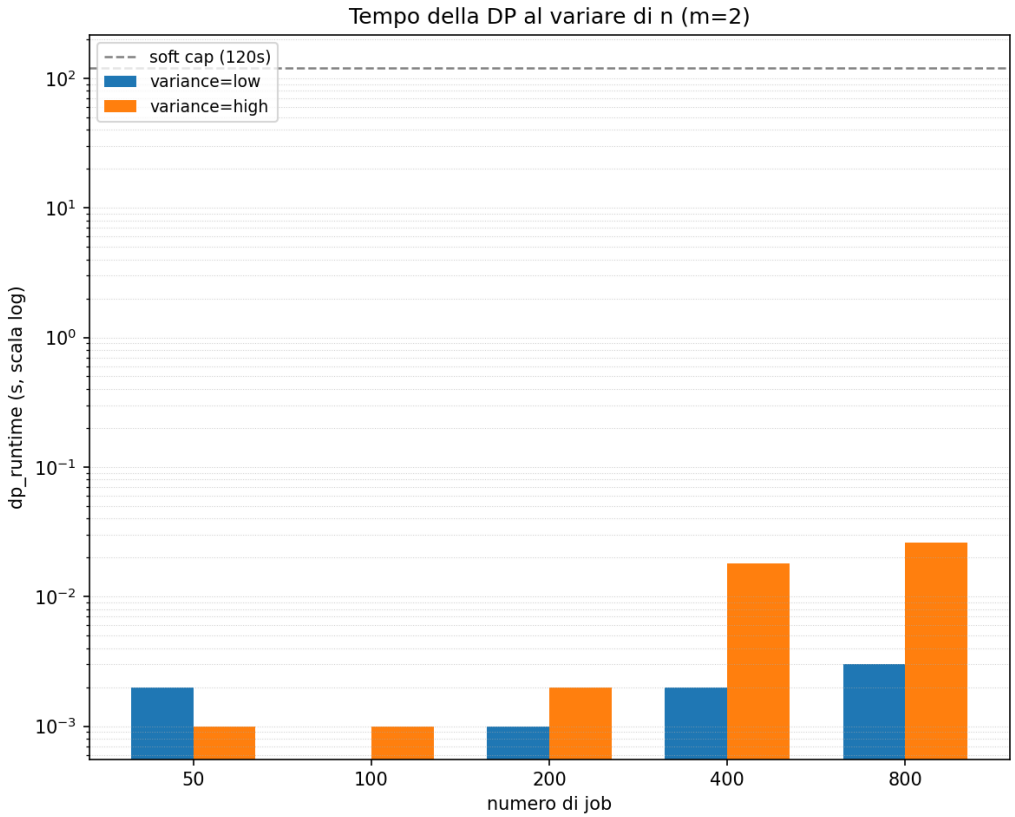
Ottimizzazione Strutturale della DP

Ristrutturazione del codice:

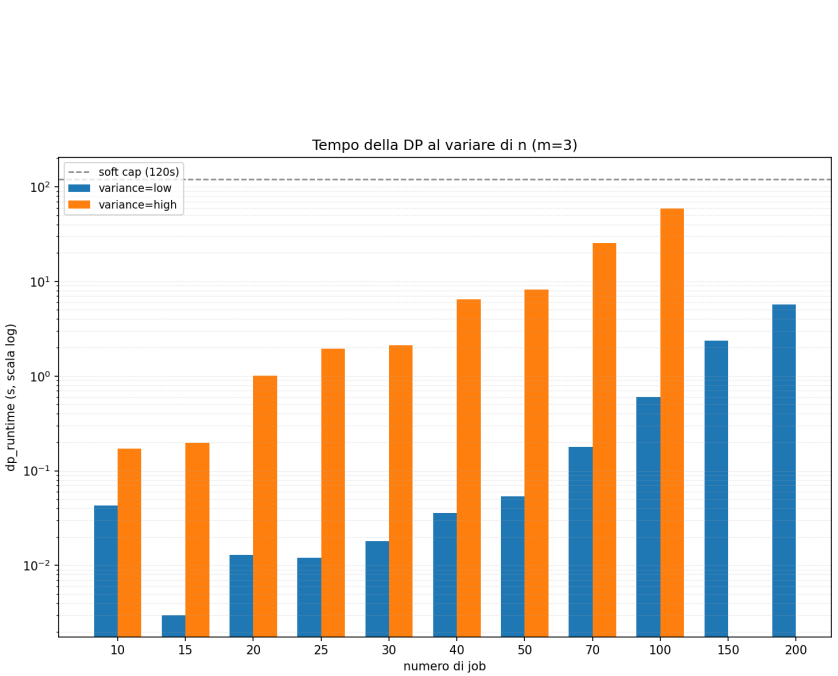
- Per mitigare i limiti prestazionali, il solutore DP seleziona dinamicamente due diverse rappresentazioni in memoria in base a m e allo spazio disponibile.
- **Rappresentazione Sparsa (Python Set):** Memorizza le tuple degli stati effettivamente raggiungibili.
  - *Pro:* Memoria contenuta quando numero di stati raggiungibili limitato; richiede meno memoria della densa per m elevati.
  - *Contro:* L'espansione degli stati richiede cicli iterativi in Python puro, risultando molto più lenta; maggiore overhead della struttura dati.
- **Rappresentazione Densa (Numpy Array):** Mappa l'intero spazio degli stati $(P+1)^{m-1}$ su un array booleano $m-1$-dimensionale.
  - *Pro:* Transizioni convertite in operazioni vettoriali di *shift* e *OR* logico eseguite in C. Velocizza i tempi di 10-50x per $m \in \{2, 3\}$.
  - *Contro:* Memoria richiesta proibitiva per $m \geq 5$; non consente backtracking della soluzione.

Ottimizzazione Strutturale della DP

Nuove Soglie per n: È stato rivalutato l'andamento dei tempi di computazione e dell'occupazione di memoria per l'algoritmo di Programmazione Dinamica. Grazie ai miglioramenti strutturali e alla vettorizzazione, la Programmazione Dinamica è riuscita a gestire istanze più grandi. Le nuove soglie per il secondo esperimento sono state elevate a $n \in \{10, 100, 200\}$.



Ottimizzazione Strutturale della DP



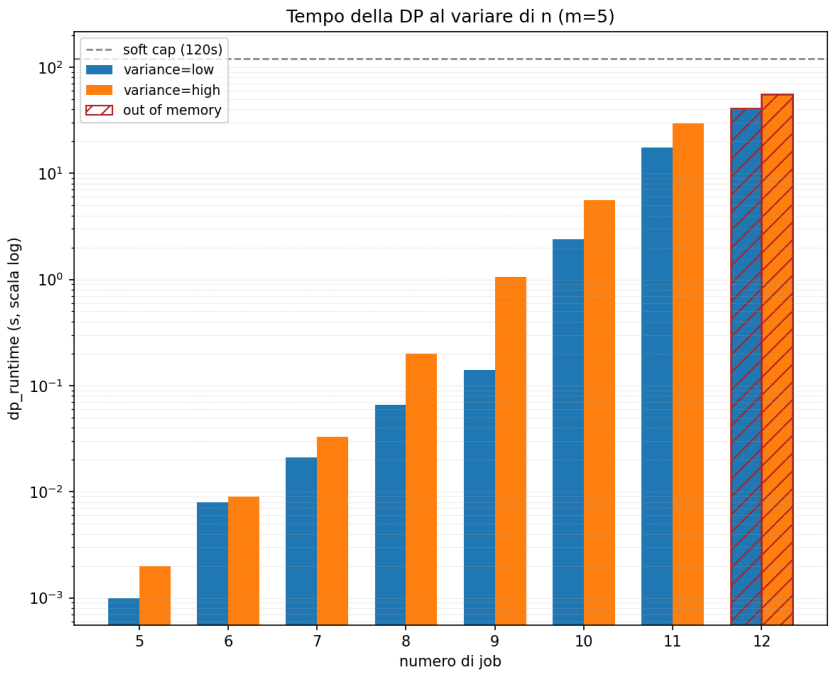


Figura: Andamento dei tempi della DP.

Secondo Esperimento di Comparazione

Andamento del Solutore Dinamico Ottimizzato: La nuova implementazione mostra una curva di crescita dei tempi molto più contenuta rispetto alla versione precedente, spostando in avanti il limite di *Out of Memory*.

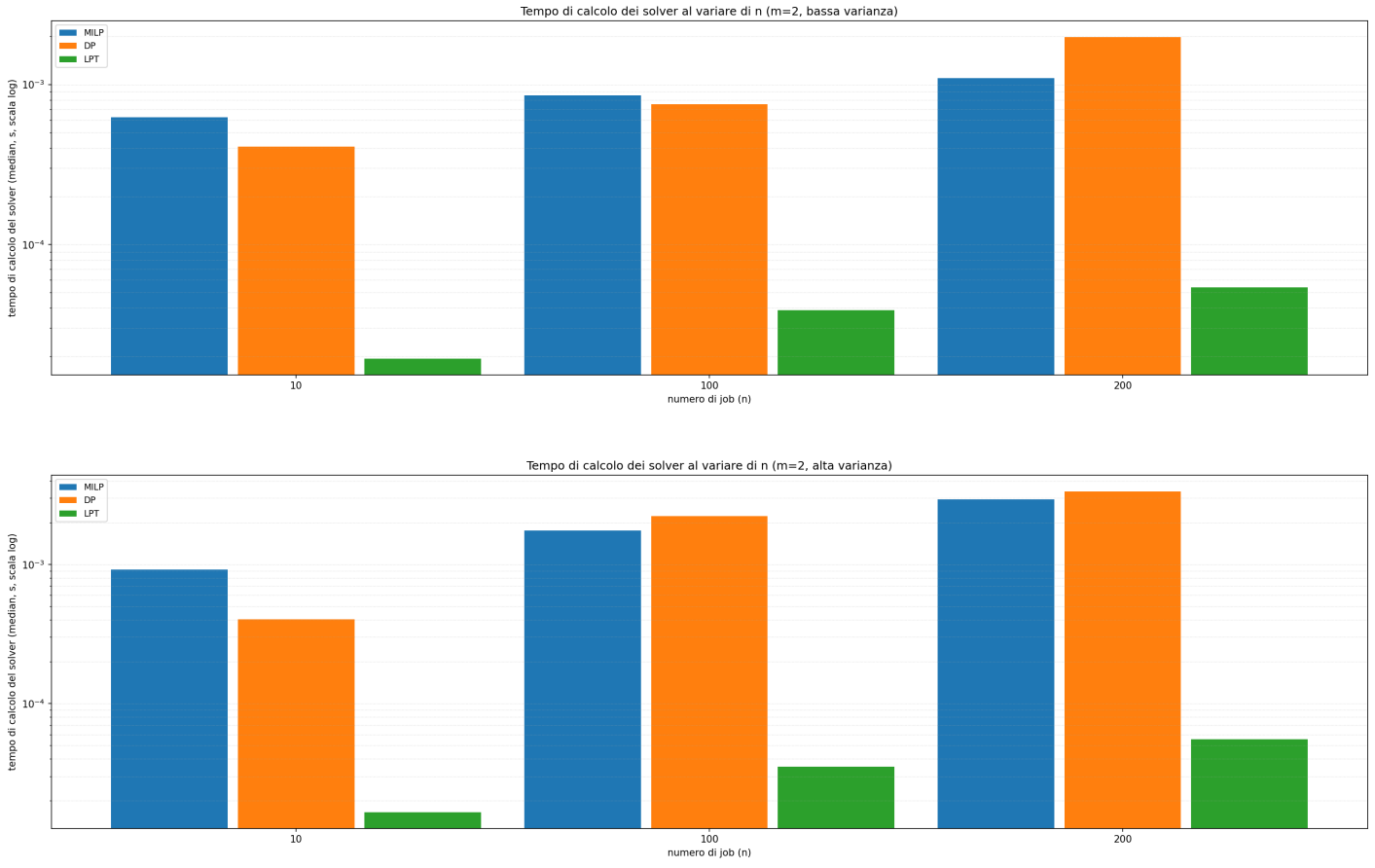


Figura: Tempi di esecuzione nel secondo esperimento, m=2.

Secondo Esperimento di Comparazione

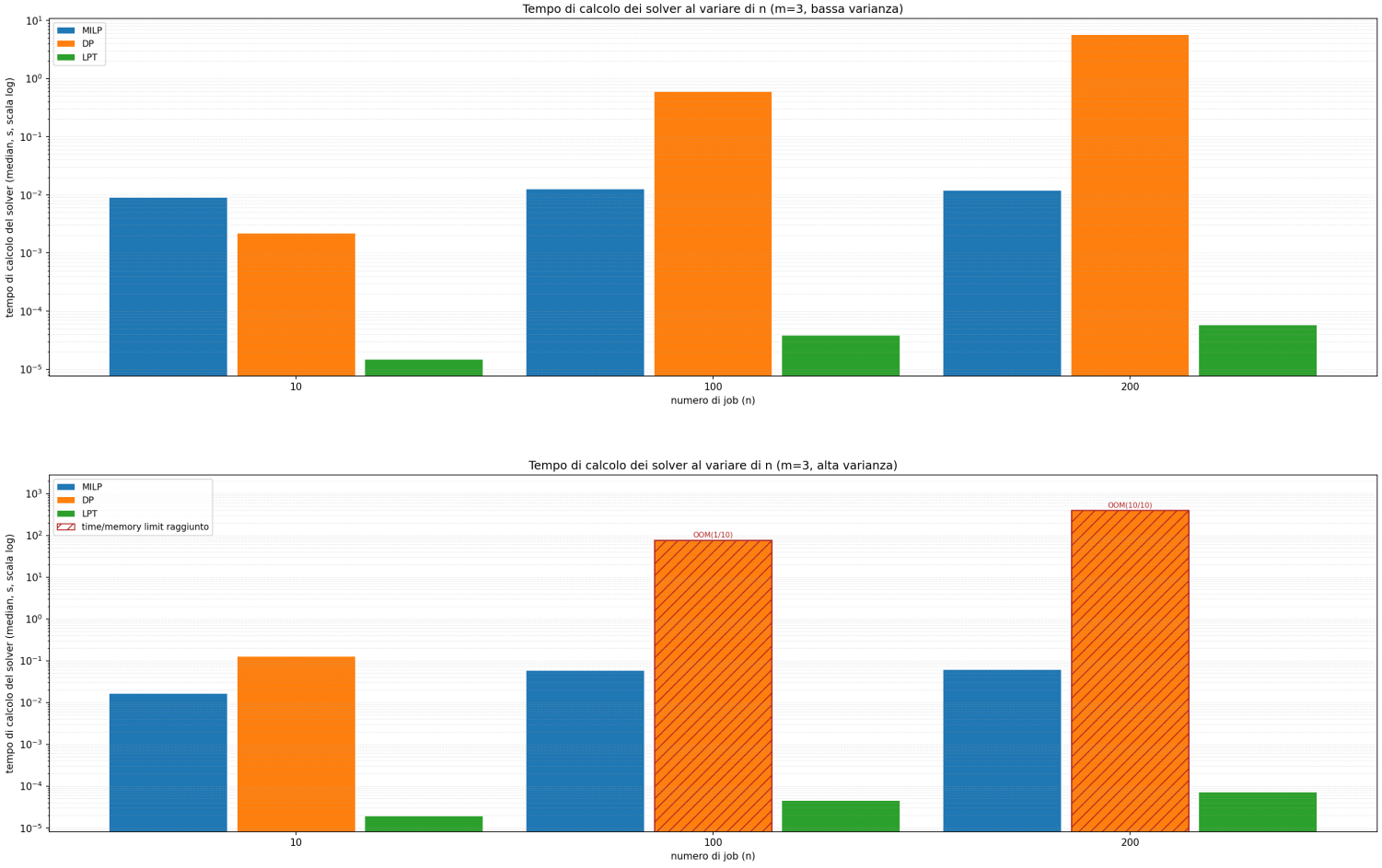


Figura: Tempi di esecuzione nel secondo esperimento, m=3.

Scalabilità della Memoria: Densa vs Sparsa ($m = 3$)

Rappresentazione Densa (Statica)

- **Struttura:** Array NumPy 2D $(P+1) \times (P+1)$ con `dtype=bool` (1 Byte/cella).
- **Indipendente da n:** Matrice pre-allocata e modificata *in-place* via shift/OR in C.

Rappresentazione Sparsa (Esponenziale)

- **Struttura:** Python `set` contenente solo le tuple raggiungibili $|R_i|$.
- **Overhead Python:** Ogni tupla richiede $\sim$ 100–150 Byte (hash, puntatori, interi).
- **Dipendente da n:** Ad *high variance*, le combinazioni esplodono.

Riscontro Sperimentale sui Dati ($m = 3$)

Exp 1: Sparsa Forzata

- $n = 25$, **alta varianza:**
  - Gli stati raggiungono fino a **70.282.611 tuple** (seed 99).
  - Prima istanza in **OOM al** 24° **job**.
- $n = 50$, **alta varianza:**
  - **10/10 istanze in OOM** attorno al job 24–28.
  - Il `set` di Python crolla per l'esplosione combinatoria delle tuple.

Exp 2: Scelta Automatica (Densa)

- $n = 100$, **alta varianza:**
  - **9/10 istanze OPTIMAL** con RAM fissa a ~ 2.5 GB.
  - *Fallback Seed 91* ($P = 54.565$): Densa stimata troppo grande $\rightarrow$ usa Sparsa $\rightarrow$ **OOM al** 26° **job**.
- $n = 200$, **alta varianza:**
  - $P > 90.000 \implies (P+1)^2 >$ 8 GB. Il solutore scarta la Densa e usa la Sparsa $\rightarrow$ **10/10 OOM** al job 24–28.

Secondo Esperimento di Comparazione

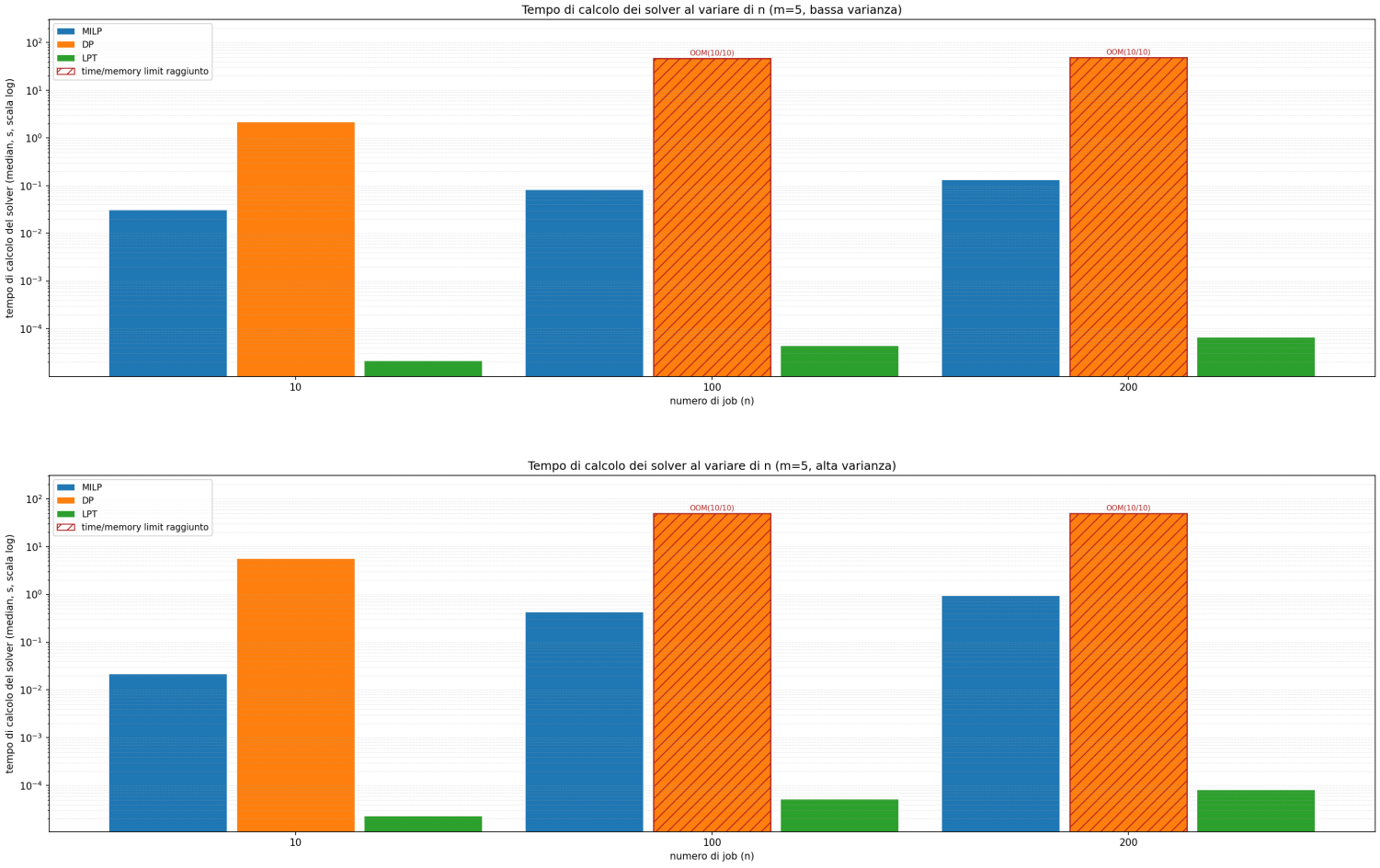


Figura: Tempi di esecuzione nel secondo esperimento, m=5.

Ottimalità delle Soluzioni

Oltre ai tempi di computazione è stata valutata anche l'ottimalità delle soluzioni fornite dai tre solutori.

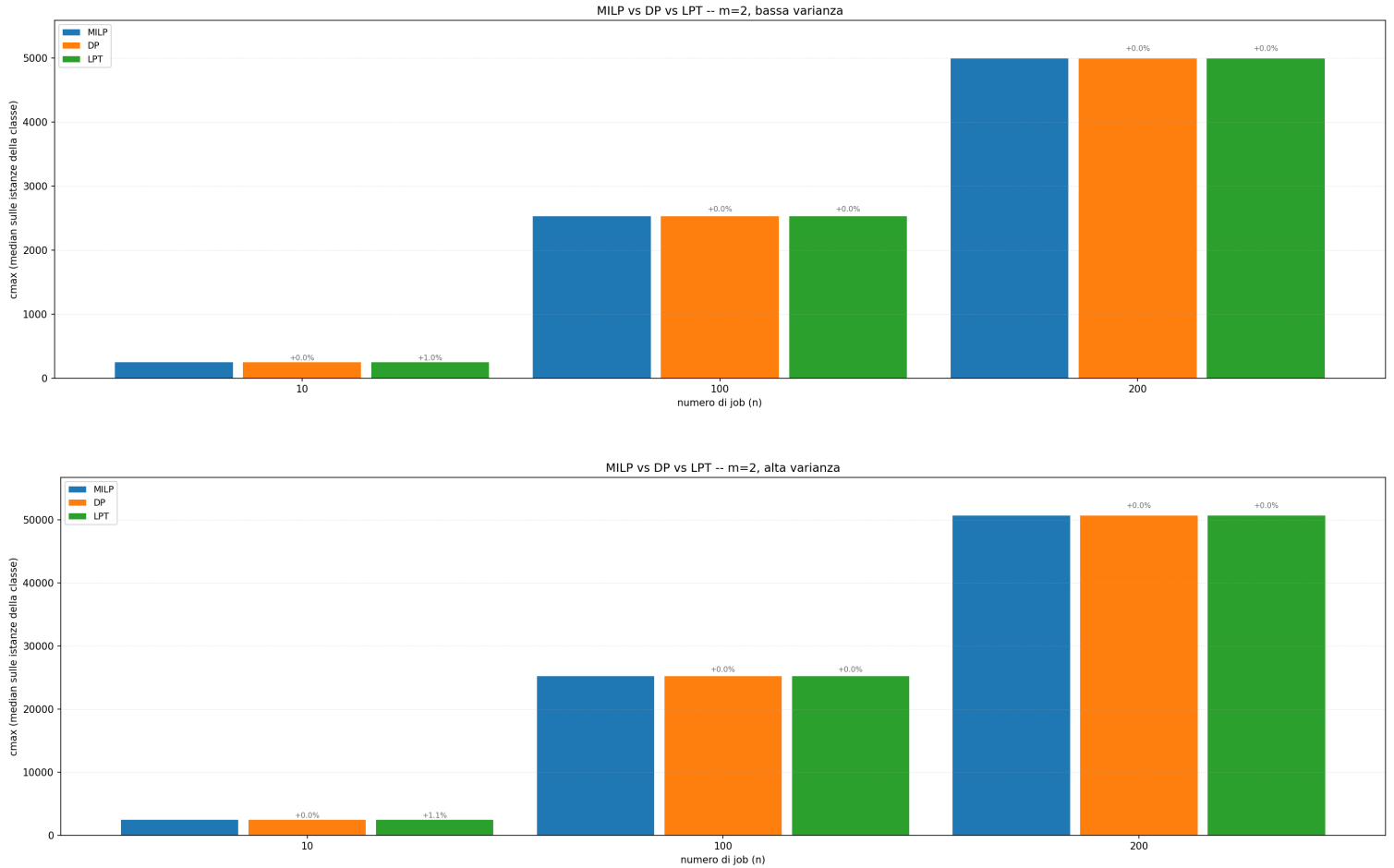


Figura: Ottimalità delle soluzioni nel secondo esperimento, m=2.

Ottimalità delle Soluzioni

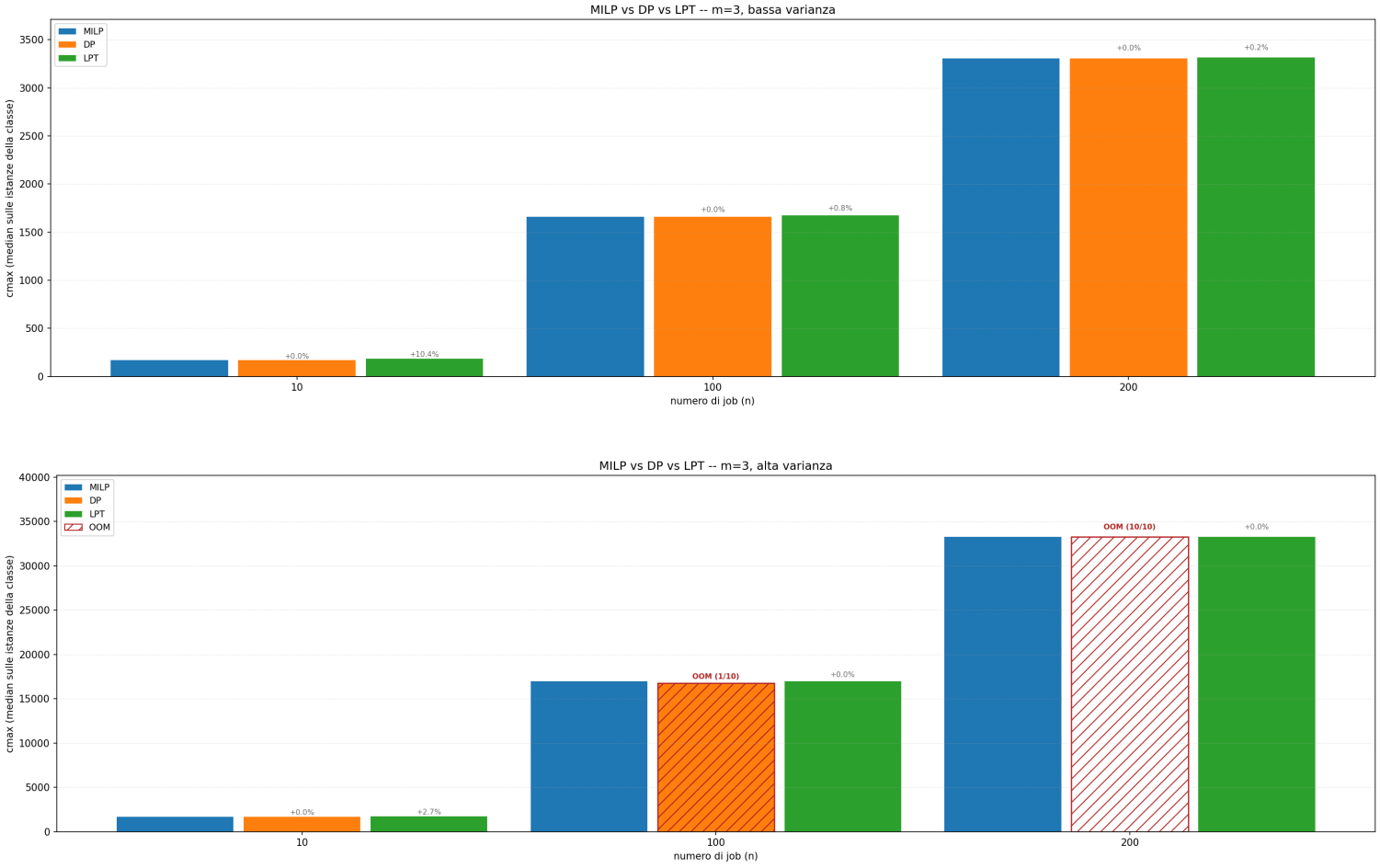


Figura: Ottimalità delle soluzioni nel secondo esperimento, m=3.

Ottimalità delle Soluzioni

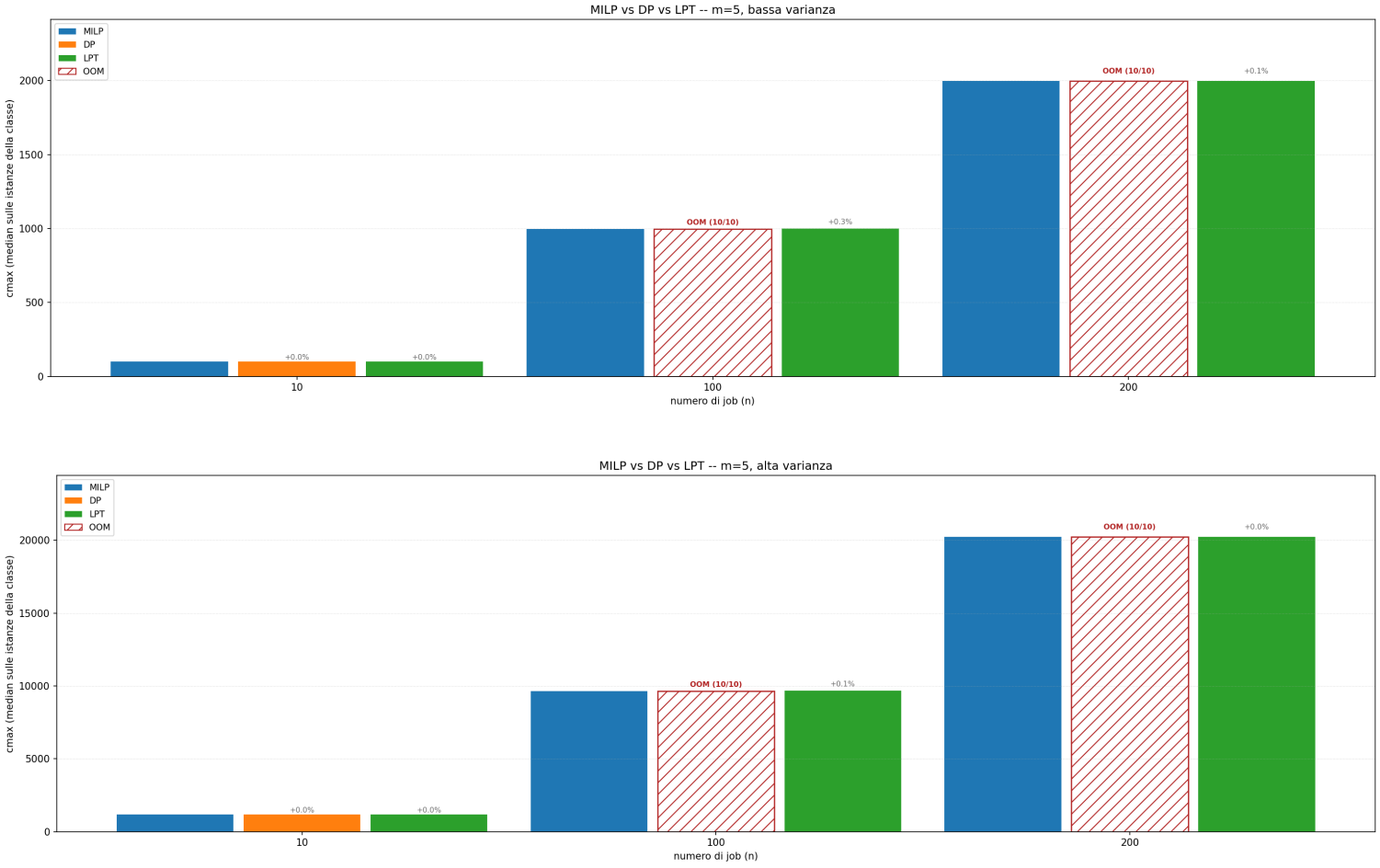


Figura: Ottimalità delle soluzioni nel secondo esperimento, m=5.

Sintesi delle Prestazioni

MIP (Esatto)

- **Robustezza:** 100% OPTIMAL (180/180 istanze, 0 time-limit).
- **Tempi:** Sub-secondo nella quasi totalità dei casi (0.6 ms per $m = 2$ fino a max ≈ 2 s per $m = 5, n = 200$).
- **Ruolo:** Metodo di riferimento per il calcolo dei gap.

DP (Dinamica)

- **Precisione:** Sempre esatta quando termina.
- **Scalabilità m:** Da pochi ms ($m = 2$) a ~ 15 s medio se completa ($m = 3$, max 85 s); se OOM, fino a ~ 470 s prima del crash.
- **Memoria ($m = 5$):** Densa non applicabile $\rightarrow$ usa Sparsa $\rightarrow$ 67% **OOM** (40/60 istanze).
- **Totale OOM:** 28% delle istanze (51/180).

LPT (Euristica)

- **Velocità:** Istantanea ($\sim \mu$s), fino a 10^7 volte più veloce della DP.
- **Andamento Gap:** Il gap medio decresce al crescere di n: 2.4% a $n = 10 \rightarrow 0.22\%$ $n = 100 \rightarrow 0.07\%$ $n = 200$.
- **Rischio:** Peak gap del **14.8%** su istanze piccole ($n = 10, m = 3$, low var).

Confronto finale

Caratteristica	MIP	DP	LPT
Garanzia di Ottimalità	100% Garantita	Garantita (solo se completa)	$R_{LPT} \leq \frac{4}{3} - \frac{1}{3m}$
Tempo di Calcolo	Stabile (0.001s–2s)	Esplode con m e P	Trascurabile (μs, costante)
Bottleneck Principale	Nessuno sulle istanze testate	Memoria RAM	Accuratezza su piccoli n
Gap medio vs Ottimo	0%	0% (se completa)	Decresce con n (2.4% $\rightarrow$ 0.07%)

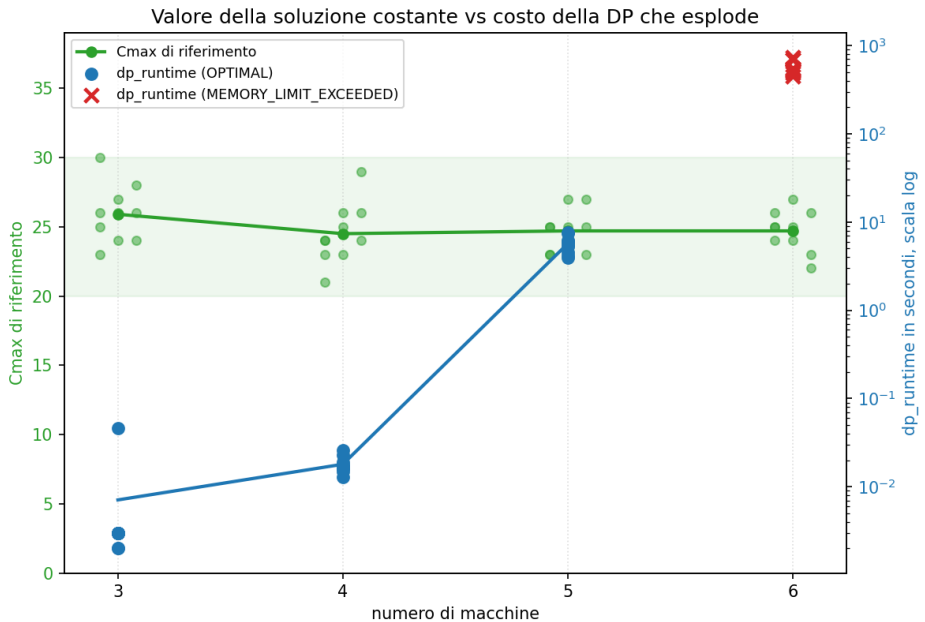
Riassumendo

- **MIP + Symmetry Breaking** è l'approccio vincente e più robusto per istanze esatte.
- **DP** soffre il limite pseudo-polinomiale: sensibile al numero di macchine m e alla dimensione massima dei carichi P (alta varianza).
- **LPT** è l'unica alternativa scalabile per contesti real-time, con scostamenti minimi su istanze grandi ma a rischio su istanze piccole.

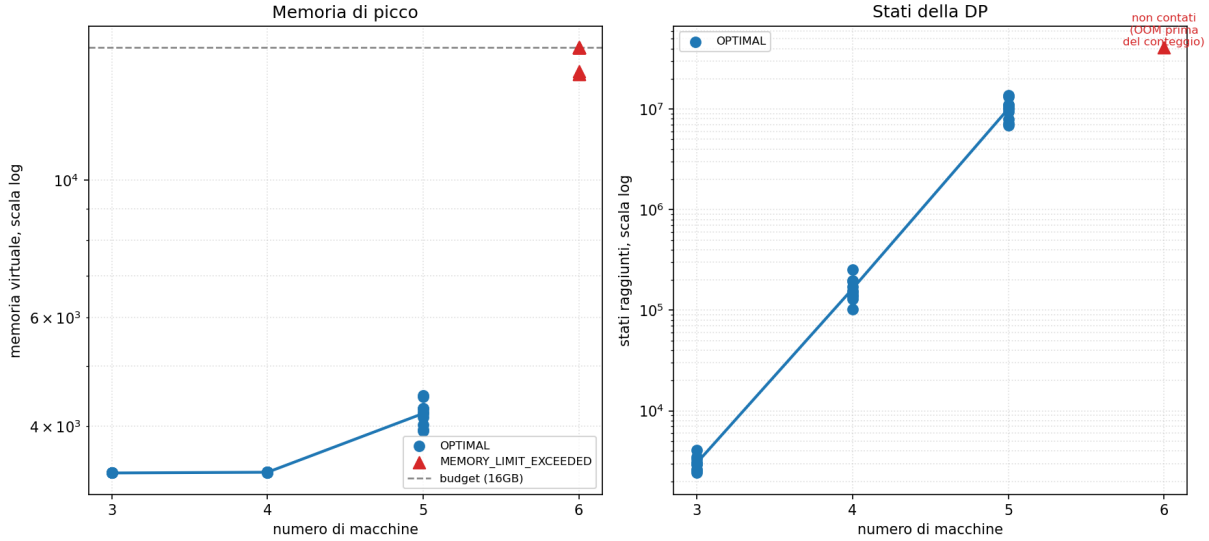
Comportamento DP con Tempi di Processamento Bassi

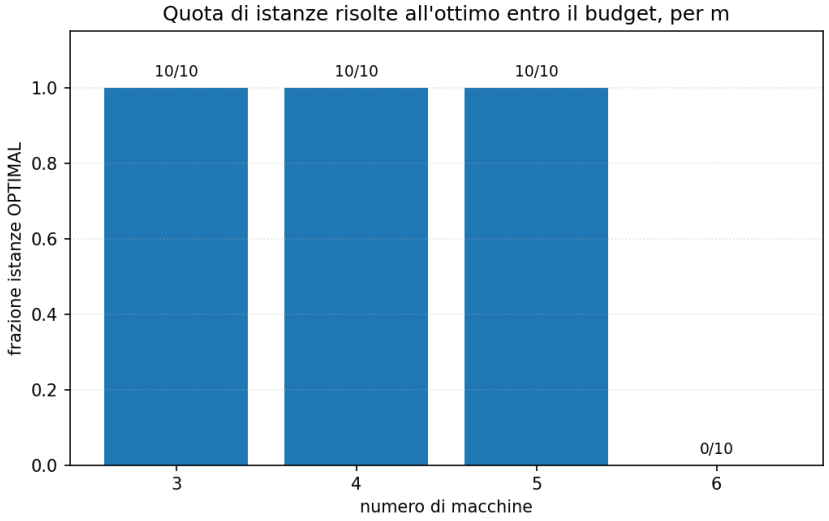
Raggiunta ottimalità con DP anche per m=5: A parità di valore della soluzione ($C_{max} \sim 25$), imponendo tempi di processamento nell'intervallo [1,5], la DP riesce, sfruttando la rappresentazione densa, a raggiungere l'ottimo anche per m=5 ($n = 42$).

Sensibilità rispetto al numero delle macchine: A parità di valore della soluzione ($C_{max} \sim 25$), ogni macchina in più costa alla DP 1-2 ordini di grandezza in più di stati/tempo/memoria.



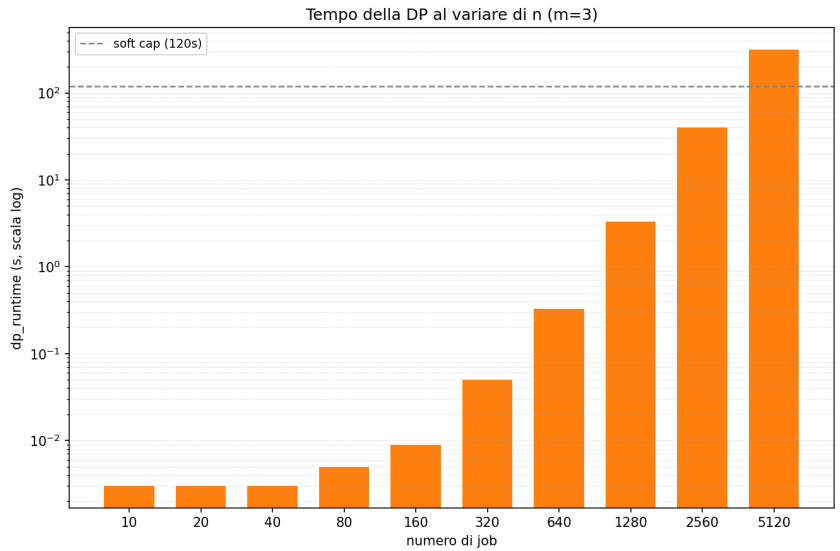
Comportamento DP con Tempi di Processamento Bassi

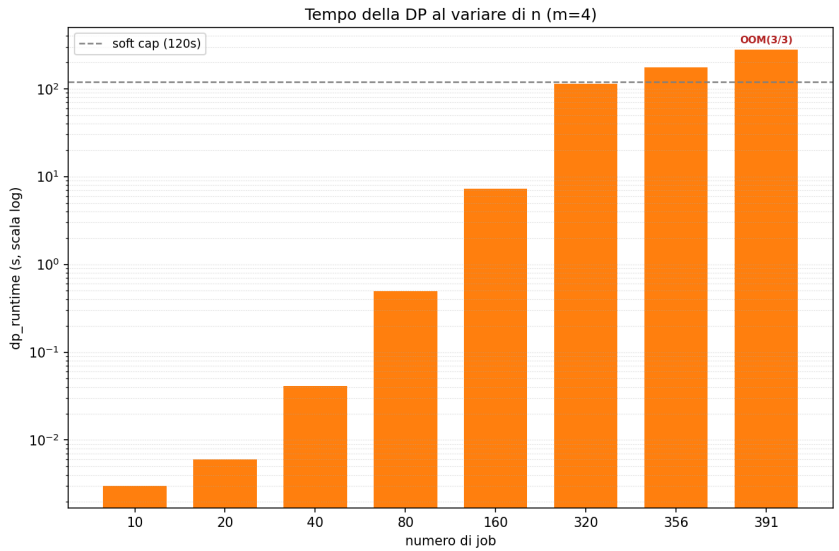




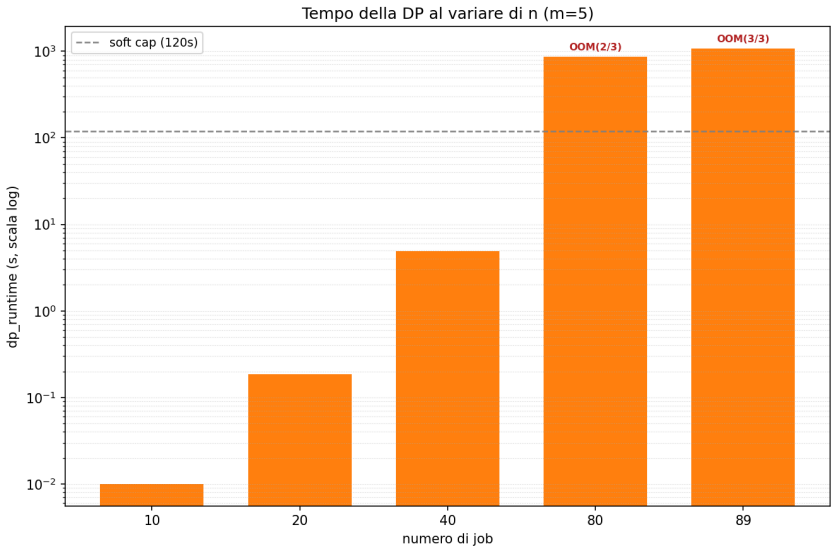
Comportamento DP con Tempi di Processamento Bassi

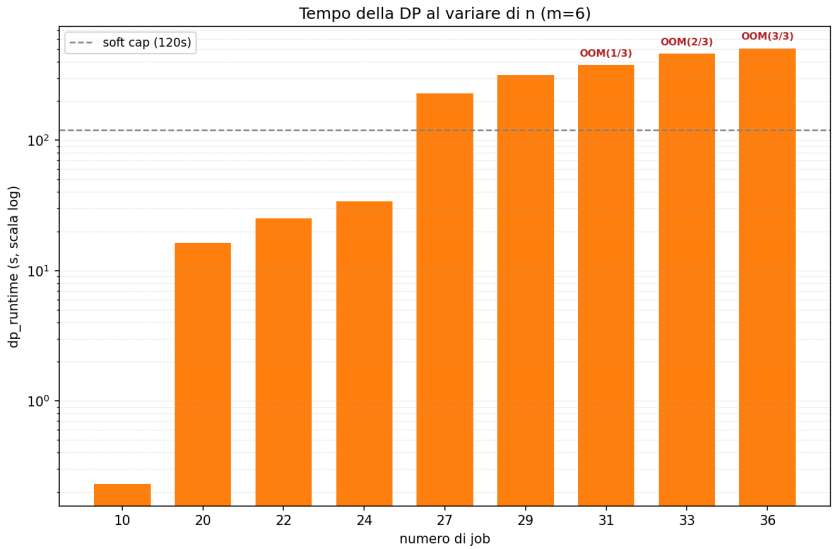
Nuove Soglie per n: È stato valutato l'andamento dei tempi di computazione e dell'occupazione di memoria per la DP. Assumendo job con tempi di processamento bassi, la DP è riuscita a gestire istanze più grandi. Le nuove soglie per la comparazione dei tre metodi sono state elevate a $n \in \{50, 225, 400\}$.





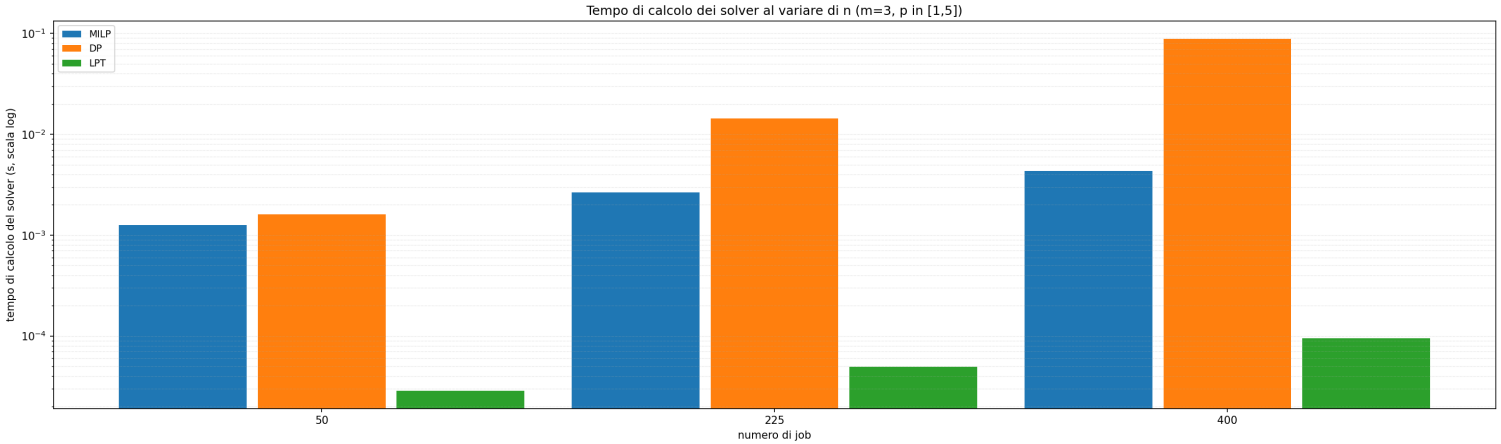
Comportamento DP con Tempi di Processamento Bassi



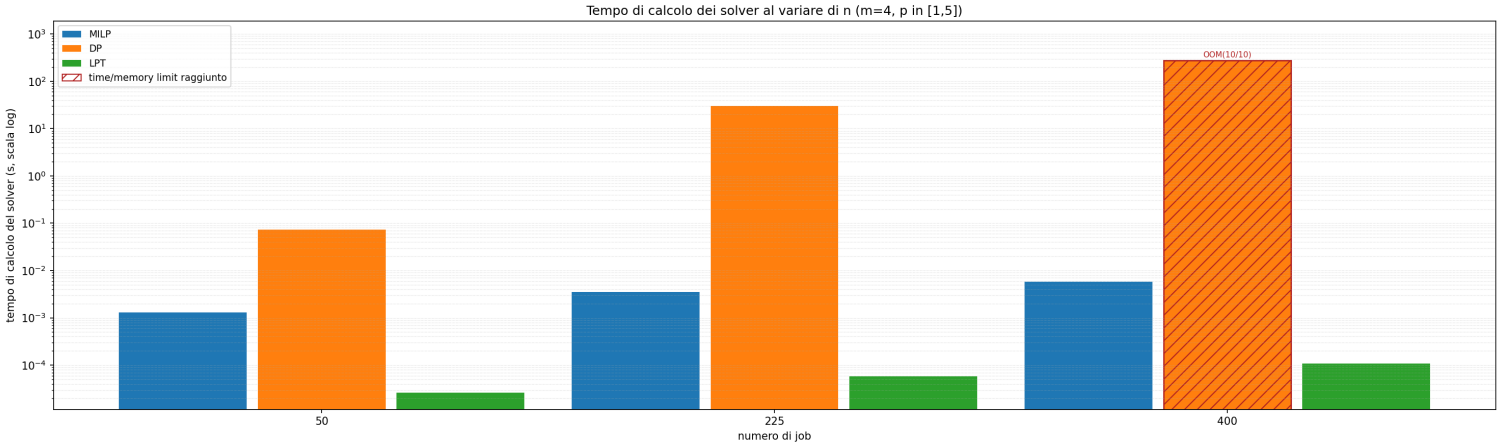


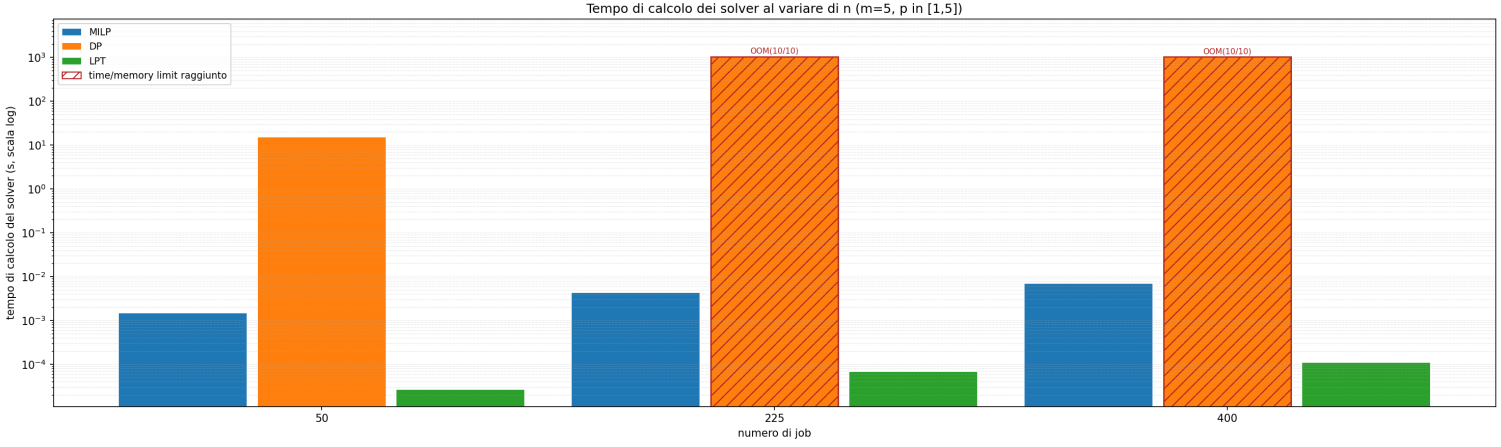
Comportamento DP con Tempi di Processamento Bassi

Andamento del Solutore Dinamico: l'impiego di tempi di processamento nell'intervallo [1,5] consente di raggiungere il limite di *Out of Memory* su istanze caratterizzate da un numero di job molto maggiore.

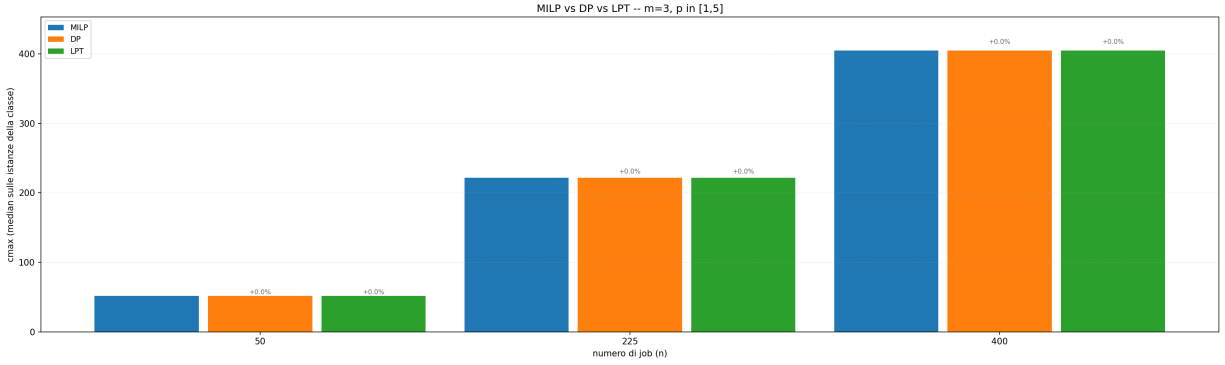


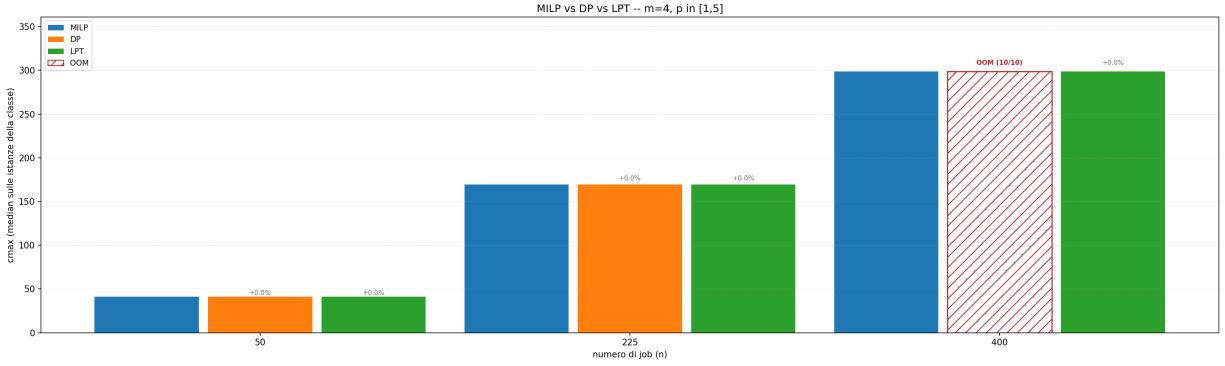
Comportamento DP con Tempi di Processamento Bassi

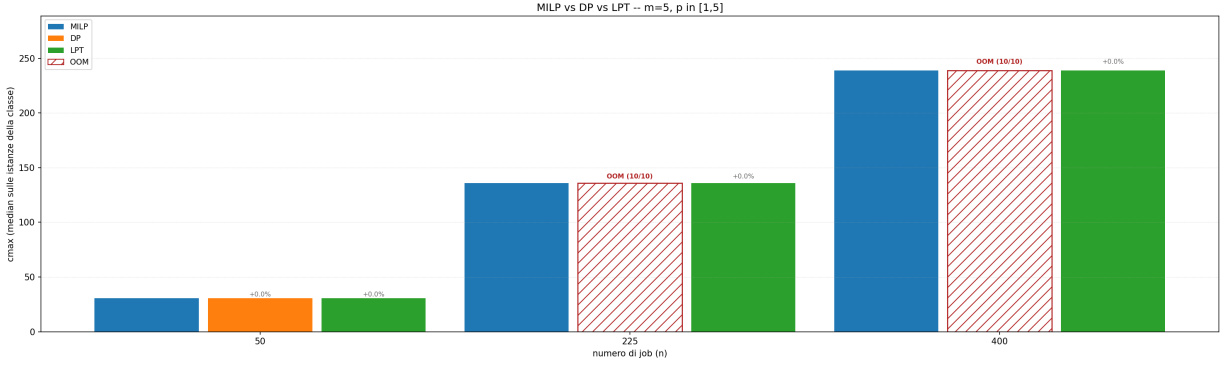




Ottimalità delle soluzioni con Tempi di Processamento Bassi







Confronto finale (tempi di processamento $p \in [1, 5]$)

Caratteristica	MIP / MILP	Programmazione Dinamica (DP)	Euristica LPT
Garanzia di Ottimalità	100% Garantita	Garantita (solo se completa)	$R_{LPT} \leq \frac{4}{3} - \frac{1}{3m}$
Tempo di Calcolo	Stabile (0.001s–0.01s)	Esplode con m	Trascurabile (20μs–39ms)
Bottleneck Principale	Nessuno sulle istanze testate	Memoria RAM (30/90 istanze in OOM, $m \geq 4$)	Nessuno sulle istanze testate
Gap medio vs Ottimo	0%	0% (se completa)	0%

Riassumendo

- **MIP + Symmetry Breaking** resta l'approccio più robusto: qui le istanze sono così leggere per Gurobi da chiudere in pochi millisecondi, sempre.
- **DP** fallisce comunque per $m \geq 4$ e n grande pur con P volutamente piccolo: conferma diretta che il collo di bottiglia è la componente esponenziale in m (dimensioni dello stato $= m - 1$), non solo P come nella campagna a varianza alta.
- **LPT** raggiunge l'ottimo su tutte le 90 istanze testate (gap 0%), non solo un'approssimazione accettabile: con molti job piccoli e omogenei il bound teorico 4/3 è estremamente pessimista rispetto al caso pratico osservato.